

alx

An Introduction to Database Systems



Train Learning Objectives



Theme

Objective

Outcome

1

Database Management

Define a database and database management system (DBMS).

- Know what a DBMS is and the role it plays between the user and the database.

2

ACID Properties

Describe the ACID properties.

- Describe each of the four acid properties.

3

Normalization

Perform database modeling and normalization.

- Know the difference between conceptual, logical, and physical ERDs.
- Be able to perform 3NF normalization.

4

DB Optimization

Describe the difference between database tuning and concurrency control.

- Describe potential database optimization methods, which include partitioning.

High-level Overview of Databases for Big Data

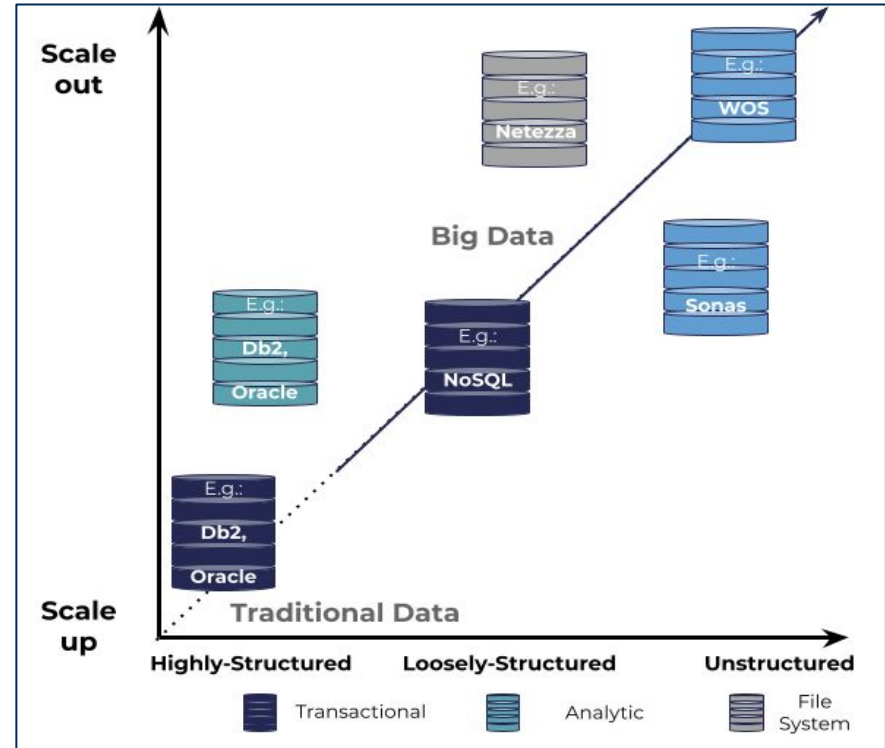
Keep information organized, accessible, and up-to-date.

Databases exist to keep large volumes of data organized and easily accessible using queries. In the context of big data, many new technologies have emerged to allow fast queries on not millions, but billions of rows of data.

In this train we will discuss database management systems used to interact with the data contained within a database, as well as the ACID properties a database needs to conform to.

We will also cover database modeling and an example of database normalization.

Finally, we will cover database optimization techniques employed in everyday enterprise-level systems.



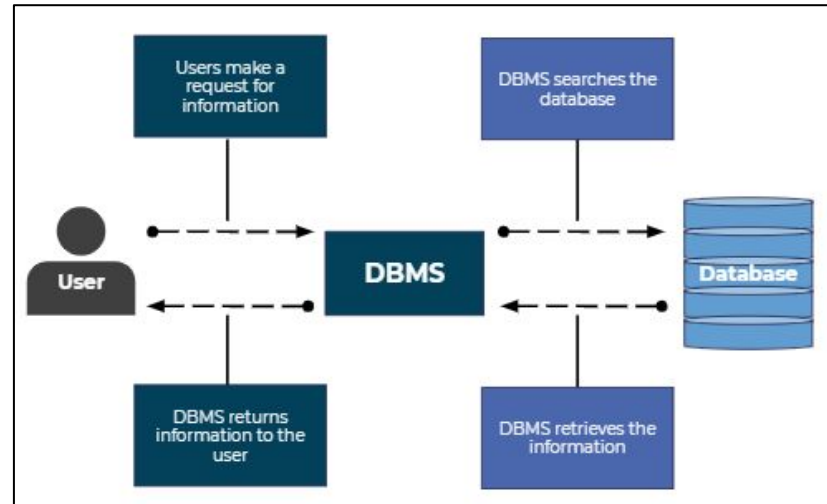
What is a Database Management System?

A database management system is a piece of software used for creating, storing, maintaining, and accessing database files.

A **Database Management System (DBMS)** is a software application for storing and retrieving users' data while maintaining appropriate security measures. It consists of a group of programs that manipulate an associated database. The DBMS accepts requests for data from an application and instructs the operating system to provide the specific data. In large systems, a DBMS helps users and other third-party software to store and retrieve data.

DBMSs allow users to create their own databases according to their requirements.

Well-known DBMSs include MySQL, MongoDB, Oracle Database, PostgreSQL, LibreOffice Base, and Microsoft SQL Server. For further information around DBMSs and related tools, [please refer to this link](#).



The Different Components of a DBMS



DBMS have different layers or components involved with the overall functioning of databases.

DBMS Component	Component Description
Software	Software is a central component to the DBMS, serving as an easy-to-use interface between the user and the database. Enables users to access, store, and modify data in the database. Interprets the database access language and translates queries into machine language. Manages the database backend, including maintenance and security.
Hardware	The actual real-world infrastructure that stores and processes the data sitting within the database. This includes servers, computers, storage devices, and I/O devices.
Data	DBMSs and databases exist to store data and would not exist without it. Data managed by a DBMS can exist as user data (transactional, operational, or any other type required by the user), metadata (data about the data in the database), and application metadata (required to run the DBMS, such as data describing the structure and format of queries and reports).
Procedures	Procedures are there to simplify the management and use of the database. Procedures are rules and instructions to help users manage the database. This includes procedures to set up and install the DBMS, to log in to the database, to create backups of the database, and to generate reports from the database. It is also possible for users to create their own stored procedures to simplify tasks that are performed regularly.
Users	Final, but also important, are users. At a high level the users include the application programmer (who designs and creates the database), database administrators (who are responsible for maintaining the DBMS and ensuring it remains secure), and end-users (actually interact with the the DBMS, storing, retrieving, adding and deleting data).

Popular DBMSs



Below is a list of the most common DBMSs that you might come across in industry. To learn more about any of the services below, click on the respective logos.



Introduction to ACID compliance



A set of properties of database transactions intended to guarantee data validity despite errors, power failures, and other mishaps.

A database operation that satisfies the **ACID properties** is called a *transaction*. ACID properties are concerned with achieving consistency and data integrity within a database. An example of this would be a person logging into their bank account. It is very important for them not to see any other person's details. The ACID properties consist of four parts:

Atomicity: Atomicity is the guarantee that an entire database operation is handled as a single entity. This means that if a single query of a database operation fails, the entire operation fails.

Consistency: Consistency within a database ensures that all database operations will only permit valid transactions. This means that a DBMS will guarantee that the database will be left in a stable and accurate state after the transaction.

Isolation: Isolation within a database guarantees that all operations will be executed independently. This means that the database will look the same after a collection of operations were executed at the same time or one after the other.

Durability: Durability means that a database will be durable enough to remember all of its queued operations, even during a system failure (for example, a restart or power outage).

Consider an application that transfers funds from one account to another:

Example

Atomicity

1



- In our example, the atomicity principle ensures that if a debit is made successfully from one account, the corresponding credit is made to the other account.

Consistency

2



- For our use case, the consistency principle ensures that the combined value of the sent/received funds is the same between the two accounts at the start and end of each transaction (the net value remains constant).

Isolation

3



- The isolation property ensures that another transaction sees the transferred funds in one account or the other, but not in both, nor in neither.

Durability

4



- In our example, the durability property ensures that changes made to each account will not be reversed.

An Overview of the End-to-End Data Modelling Process



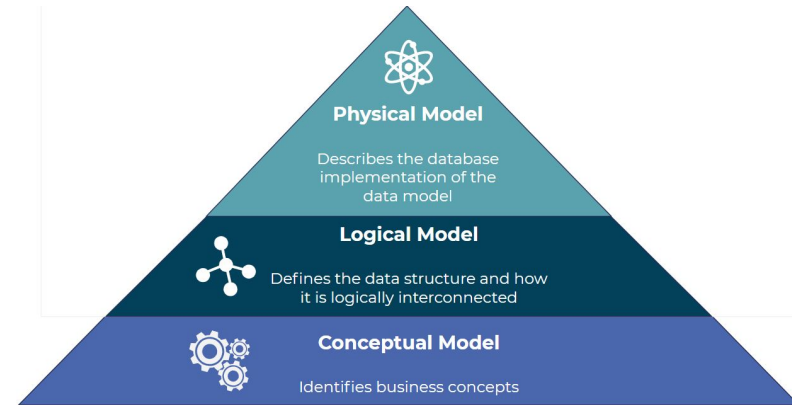
Before we can start storing, querying, and manipulating data, we need to design the database in a process known as data modelling. We discuss this process in the context of relational databases within the following slides.

Data modeling is the set of database design steps taken to create a conceptual representation of objects stored within a database and the relationships between them.

Data modeling engages multiple levels of a business in order to correctly define and enforce business rules, regulatory compliances, and government policies on the data. This ensures that the database requirements are met and scoped out before moving to development. Once a relational database grows and becomes available in a production environment, changes may be much more difficult to implement.

The three phases of data modelling include:

- I. Fully describing the needs of whoever will use the database (conceptual).
- II. Choosing a data model (logical).
- III. Moving from an abstract data model to the implementation of the database (physical).



What is an Entity Relationship Diagram, and how is it used in data modeling?



An **Entity Relationship Diagram** (ERD) is a diagram or “flowchart” of a database. They are primarily used to design the database during the modeling phase but are also used by business managers who potentially want to expand the number of features contained in the database or for educating new members of the team. For a *relational database*, an ERD consists of entities, attributes, and relationships:



Entities: Entities are tables within a database. A database of a school could potentially contain entities for learners, teachers, and subjects. A single row within a table is known as an instance.



Attributes: These are forms of information that further define the entities. Within a table, these would be the column headings. For example, a student table might contain name, surname, grade, age, and subject columns.



Relationships: Relationships define how entities interact. These are the primary and foreign key relationships between tables.

Phase One of Data Modelling: Conceptual

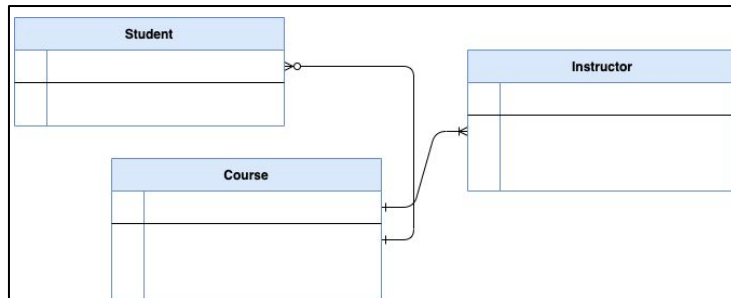
Overview of the conceptual data model.

Conceptual Data Model: Understanding User Needs

Understanding user needs forms part of the initial phase of data modeling and includes, amongst other activities, discussions with the prospective database owners regarding what data they need available, the structure of the data they want available, how often they want it available, and how accessible they need it to be within their company or organization.

This process is known as requirement analysis, and is usually performed by an expert in the field of database design. This expert will have frequent discussions on the needs and requirements of the database with someone more experienced in the domain.

Conceptual ERDs (such as the one pictured on the right) **model** the entities that should exist in a database and the relationships between them. No attributes are defined yet.



An example of a conceptual ERD showing the high-level design of our relational database. During this **initial phase** of data modeling, we should only be concerned with the **tables** in our database and the **relationships** between them.

In this example:

1. Many students can enroll in one course.
2. One or more instructors can invigilate one course.

Phase Two of Data Modelling: Logical



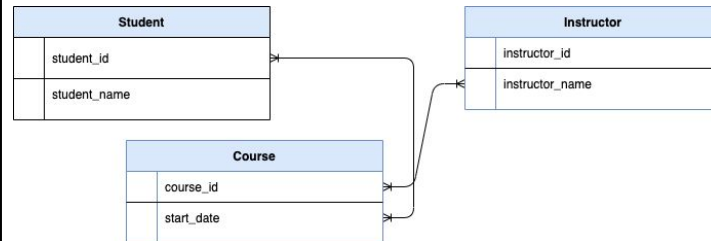
Overview of the logical data model.

Logical Data Model: Adding the Details

The **Logical Data Model** expands on the conceptual data model by *defining the attributes of each entity*. This model is most notably concerned with the business needs, and deals with the conversion of a conceptual model into a schema that supports the business needs.

Characteristics of a logical data model include:

- Describing the business model/needs well enough to develop it into an **understandable schema**.
- The model is designed to be **DBMS agnostic and** is generic.
- The model is usually **normalized to 3NF**.



Our example model has now expanded to include attributes for all entities:

Student now contains a 'student_id' and 'student_name'

Course now contains a 'course_id' and 'start_date'

Instructor now contains an 'instructor_id' and 'instructor_name'

Phase Three of Data Modelling: Physical



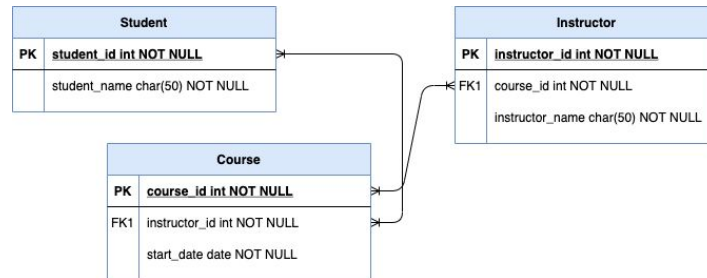
Overview of the physical data model.

Physical Data Model: The Final Form

A **Physical Data Model** represents the finalised, implementable relational database.

It contains all the attributes, specified by type. The relationships are also well defined in the physical model, and include primary- and foreign key relationships between all database entities.

Methods to implement the model on a specific DBMS (MySQL as an example) should be noted, preferably with a version used as well. This ensures that as much information as needed is included to recreate a working version of the database using the physical ERD.



Our example model has now expanded to include:

- *Primary and foreign key relationships* between the different entities.
- *Fully defined attributes* including type and nullability.

Database Design and Modelling: Conceptual

To recap the data modelling phases, we provide a simple student database example below:

Conceptual

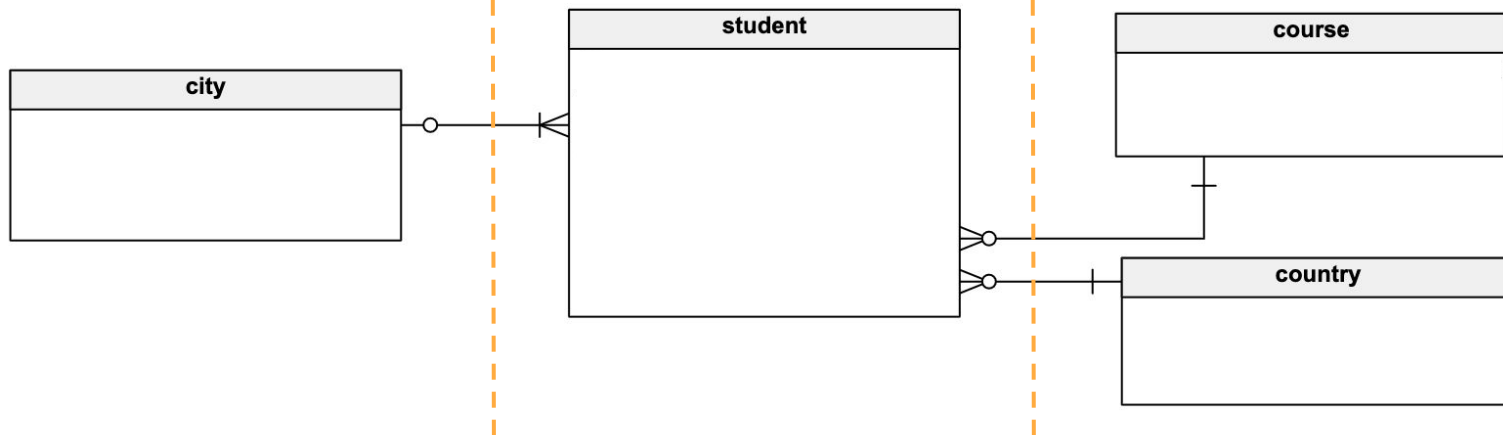
- Entity (table) names
- Relationships

Logical

- Attribute (column) names
- Database is normalized (3NF)

Physical

- Attribute types added
- Primary keys are highlighted
- Designed around specific DBMS taken into account

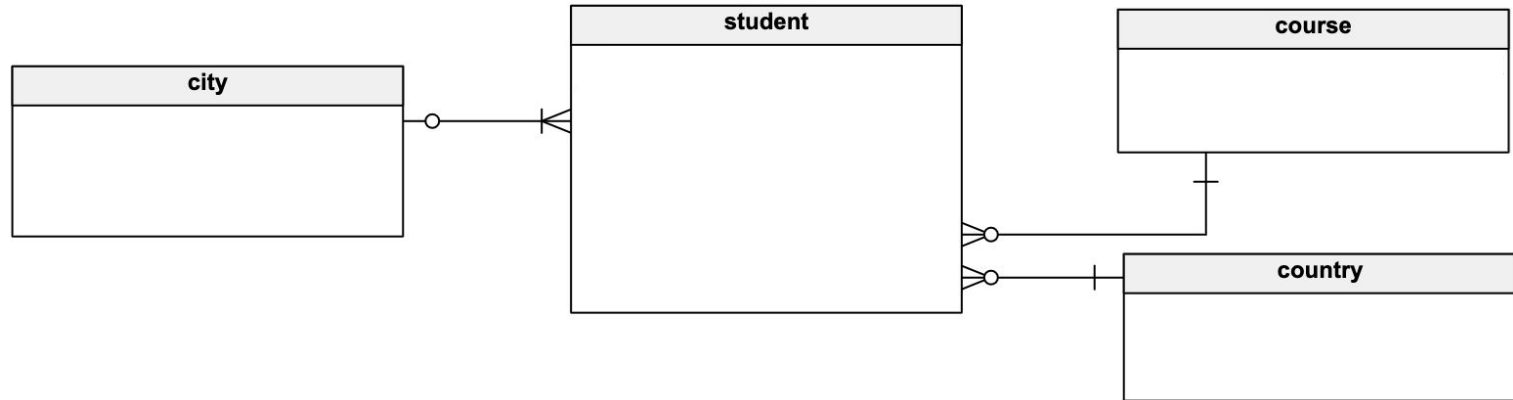


Database Design and Modelling: Conceptual



To recap the data modeling phases, we provide a simple student database example below:

The **Conceptual Data Model** is where the entities that should exist in a database and the relationships between them are defined.

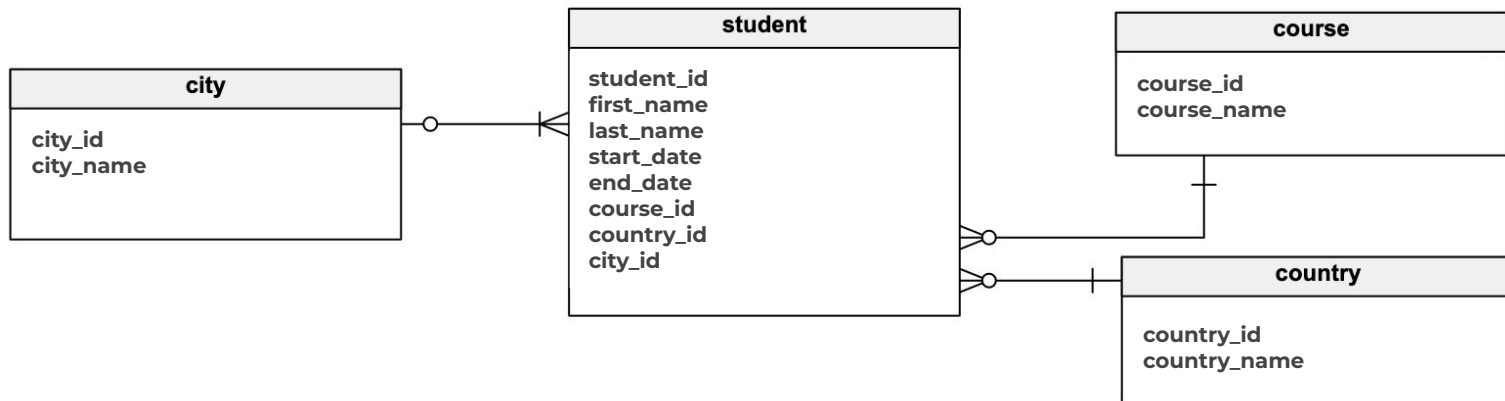


Database Design and Modelling: Logical



To recap the data modelling phases, we provide a simple student database example below:

The **Logical Data Model** expands on the conceptual data model by *defining the attributes of each entity*. This model is most notably concerned with the business needs, and deals with the conversion of a conceptual model into a schema that supports the business needs.

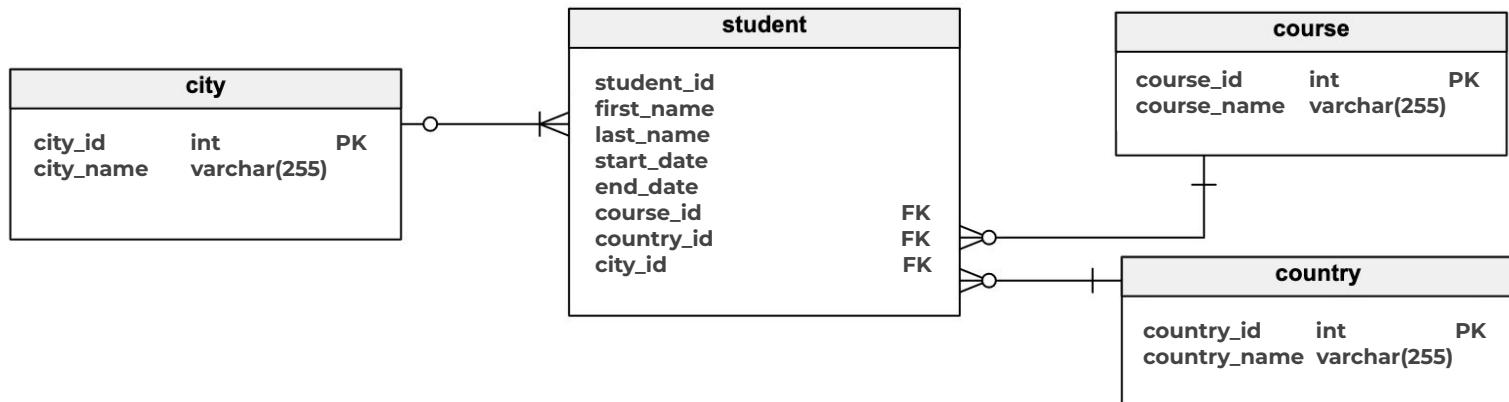


Database Design and Modelling: Physical



To recap the data modelling phases, we provide a simple student database example below:

A **Physical Data Model** represents the finalised, implementable database. It contains all the attributes, specified by type. The relationships are also well defined in the physical model, and include primary- and foreign key relationships between all database entities.



A Brief Overview of Normalization



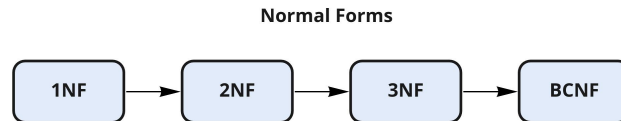
For a brief overview of normalization, have a read-through [this](#) resource

Normalization is the process of organizing the data in a database, and is typically applied to relational databases. It is used to minimize the redundancy from a relation or set of relations. It is also used to eliminate the undesirable characteristics including *insertion*, *update* and *deletion* anomalies. Normalization is performed by dividing a larger table into smaller tables, linked using relations.

A relational database is considered “normalised” once it is in 3NF.

There are 4 types of normal forms:

1. A database is in 1NF if it contains only atomic values, meaning attributes of tables can't hold multiple values. [Example](#).
2. A database is in 2NF if it is in 1NF and all non-key attributes are *fully functional dependent* on the primary key. [Example](#).
3. A database will be in 3NF if it is in 2NF, and no transitive dependencies exists between its relations. [Example](#).
4. Otherwise known as Boyce Codd normal form (BCNF), this occurs when a database has no *multivalued dependency*. [Example](#).



A Worked Normalization Example



Normalizing to Third Normal Form

Normalization to 3NF example:

A relational database is in third normal form if it satisfies the following conditions:

- It is in second normal form.
- There are no transitive functional dependencies between its relations.

Consider the table below, "table_book_detail".

In the table, *[Book ID]* determines *[Genre ID]*, and *[Genre ID]* determines *[Genre Type]*. Therefore, *[Book ID]* determines *[Genre Type]* via *[Genre ID]*, thus giving a transitive functional dependency. The resulting structure does not satisfy Third Normal form.

Table book_detail

Book ID	Genre ID	Genre Type	Price
1	1	Travel	9.99
2	1	Travel	17.99
3	2	Business	4.99
4	3	Religion	14.99
5	1	Travel	12.99

A Worked Normalization Example



Normalizing to Third Normal Form (cont.)

Normalization to 3NF example:

To normalize this table to third normal norm, we split the table into two separate entities as follows:

Now all non-key attributes are fully functional dependent only on the primary key. In $[TABLE_BOOK]$, both $[Genre\ ID]$ and $[Price]$ are only dependent on $[Book\ ID]$. In $[TABLE_GENRE]$, $[Genre\ Type]$ is only dependent on $[Genre\ ID]$.

Table book

Book ID	Genre ID	Price
1	1	9.99
2	1	17.99
3	2	4.99
4	3	14.99
5	1	12.99

Table genre

Genre ID	Genre Type
1	Travel
2	Business
3	Religion

General Relational Database Optimisation Techniques



Building a production-ready database can be very rewarding, but once a large number of people start using it, it may start to slow down unexpectedly. Here are some ways to restore efficiency:

1. **Good database design:** As the size of your database grows, so too do the number of relationships between entities. This will lead to more and more computationally expensive joins being needed to retrieve data through these relationships. In many cases a solution can be to iterate on our database design by simplifying the relationships between entities or to use a different database type (for example, NoSQL).
2. **Disk I/O optimization:** The number of times a storage medium (hard drive or RAM) can be read to and from per second is limited by physics. By increasing the query batch size within our DBMS (allowing larger chunks of queries to be processed simultaneously), we can prevent bottlenecks around our storage medium's I/O limit—thereby increasing throughput.
3. **Database checkpointing control:** Checkpointing within a DBMS occurs when unused or “dirty” cache is cleared. This process takes up a large portion of your storage medium's I/O limit and decreases the database's performance, albeit temporarily. By scheduling our DBMS's checkpointing routine to only run when a low number of users are using the database, we can increase performance.
4. **Concurrency control:** This forms part of the disk I/O optimization technique and takes advantage of a modern CPU's ability to run thousands of concurrent threads. By enabling or tuning this feature within a DBMS, we can run multiple batches of queries simultaneously.

Concurrency Control for Database Optimisation and Procedure Management

Of the optimization techniques we've just discussed, the first three are implemented for performance improvements that are beneficial but not crucial. However, when a multi-user database with both read and write actions is created, concurrency control is necessary for it to function properly.

Consider the following example: Two people visit a hotel booking website at an identical time to book a room - for the same night at the same hotel. However, there is only one room left in the hotel for that particular evening. Here, without concurrency control in DBMS, it is possible that both people will end up purchasing the same hotel room. This is where concurrency control methods are vital, as they do not allow this scenario to happen.

Concurrency control methods include the following:

- **Lock-Based Protocols:** These methods 'lock' entities while they are being accessed, not allowing a database operation to take place unless the previous transaction has completed.
- **Timestamp-Based Protocols:** This method ensures that database operations trying to access the same data will do so in the order that they were executed.
- **Validation-Based Protocols:** Database operations performed with this concurrency control method do not update the database itself, but rather only a local version of the database. Once an operation is completed, its resulting local version is compared against the main version and is merged if no discrepancies are detected.

Optimising Database Performance with Partitioning and Sharding



Partitioning and sharding are closely related techniques used to optimize database performance—especially when working in high-throughput environments.

Database partitioning is a modern database optimization technique. In the context of relational databases, partitioning sees larger tables split into smaller ones to be processed by different processor threads simultaneously.

This partitioning can be divided into two types:

Vertical Partitioning: occurs when the smaller tables each contain a **subset** of the original table's **attributes (columns)**. I.e., the number of rows is constant across the tables, but the individual rows are divided.

Horizontal Partitioning: occurs when the smaller tables each contain a **unique subset** of the original table's **rows**. I.e., the table schema remains constant, but the individual rows are divided.

When either method of splitting is **used on a single machine**, it is referred to as **partitioning**. When the splitting is performed **over multiple machines**, it is known as **sharding**.

Historically, partitioning was used to exploit the powerful multi-threaded CPUs available to industry. As data sources continued to grow, however, sharding (horizontal partitioning) was introduced, allowing tables to be partitioned (into shards) across multiple machines. Individual shards are also smaller than the original table, further reducing the number of rows to be processed by a machine during a query.

Optimizing Database Performance with Partitioning and Sharding (Cont.)

Horizontal sharding provides the added benefit of increasing the resilience of our database. If an outage of one of the machines were to happen, only the chunk of data present on that machine would be lost temporarily. The remaining shards of the table would still operate and return queries successfully.

An example of partitioning and sharding can be seen below:

- **Vertical partitioning:** The original student table is split into two smaller ones with different attributes (name vs. location details).
- **Horizontal partitioning (sharding):** The smaller tables contain all of a student's details, but each table contains different students.

Student ID	First Name	Last Name	City
1	Thabo	Makuse	Cape Town
2	Ben	Howe	Pretoria
3	Tom	Laste	Durban
4	Sipho	Bushane	Cape Town

Original table

Student ID	First Name	Last Name
1	Thabo	Makuse
2	Ben	Howe
3	Tom	Laste
4	Sipho	Bushane

**Vertically
partitioned table**

Student ID	City
1	Cape Town
2	Pretoria
3	Durban
4	Cape Town

Student ID	First Name	Last Name	City
1	Thabo	Makuse	Cape Town
2	Ben	Howe	Pretoria

Student ID	First Name	Last Name	City
3	Tom	Laste	Durban
4	Sipho	Bushane	Cape Town

**Horizontally
partitioned table**

Conclusion



In this train, we covered the following:

- **We gained an understanding of what a DBMS is** and its importance in facilitating database operations.
- We received an **overview of the ACID properties** required **for successful database transactions**;
- We were introduced to **relational database modelling** and conceptual, logical and physical ERD diagrams;
- We covered the theory and a short example around **relational database normalization**;
- We considered **database optimization techniques**, including database tuning methods and concurrency control.
- Finally, we explored **database partitioning and sharding** (horizontal partitioning).